

Cloud Hypervisor + GUI - Complete Setup Guide

Goal: Run Firefox browser inside Cloud Hypervisor microVM with GUI via VNC/SPICE

Requirements:

- Ubuntu 22.04 LTS or later
 - KVM support (Intel VT-x or AMD-V)
 - 8GB RAM minimum
 - Rust compiler (will install)
-

Part 1: System Preparation

Step 1.1: Verify Virtualization Support

```
bash

# Check CPU virtualization
egrep -c '(vmx|svm)' /proc/cpuinfo
# Should return number > 0

# Check KVM availability
ls -l /dev/kvm
# Should exist with rw permissions

# Load KVM modules if not loaded
sudo modprobe kvm
sudo modprobe kvm_intel # OR: sudo modprobe kvm_amd
```

Step 1.2: Install Dependencies

```
bash
```

```
# Update system
sudo apt update && sudo apt upgrade -y

# Install build dependencies
sudo apt install -y \
    build-essential \
    git \
    curl \
    pkg-config \
    libssl-dev \
    libglib2.0-dev \
    libpixman-1-dev \
    libcap-ng-dev \
    qemu-utils \
    cloud-image-utils \
    genisoimage \
    virt-viewer \
    spice-client-gtk

# Install Rust (Cloud Hypervisor is written in Rust)
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source $HOME/.cargo/env

# Verify Rust installation
rustc --version
cargo --version
```

Part 2: Build Cloud Hypervisor

Step 2.1: Clone and Compile

```
bash
```

```
# Create working directory
mkdir -p ~/cloud-hypervisor-setup
cd ~/cloud-hypervisor-setup

# Clone Cloud Hypervisor repository
git clone https://github.com/cloud-hypervisor/cloud-hypervisor.git
cd cloud-hypervisor

# Build (this takes 5-15 minutes)
cargo build --release

# Install binary
sudo cp target/release/cloud-hypervisor /usr/local/bin/
sudo chmod +x /usr/local/bin/cloud-hypervisor

# Verify installation
cloud-hypervisor --version
# Should show: cloud-hypervisor v38.0 or similar

cd ~/cloud-hypervisor-setup
```

Part 3: Create Guest OS Image

Step 3.1: Download Ubuntu Cloud Image

```
bash

cd ~/cloud-hypervisor-setup

# Download Ubuntu 22.04 cloud image
wget https://cloud-images.ubuntu.com/jammy/current/jammy-server-cloudimg-amd64.img

# Resize image to add more space
qemu-img resize jammy-server-cloudimg-amd64.img +10G

# Create working copy
cp jammy-server-cloudimg-amd64.img ubuntu-browser.img

echo "✅ Base image downloaded and resized"
```

Step 3.2: Create Cloud-Init Configuration

```
bash
```

```
cd ~/cloud-hypervisor-setup
```

```
# Create user-data file for cloud-init
```

```
cat > user-data << 'EOF'
```

```
#cloud-config
```

```
# Set hostname
```

```
hostname: browser-isolation
```

```
# Create user with password
```

```
users:
```

```
- name: browser
```

```
  sudo: ALL=(ALL) NOPASSWD:ALL
```

```
  groups: users, admin
```

```
  home: /home/browser
```

```
  shell: /bin/bash
```

```
  lock_passwd: false
```

```
  passwd: $6$rounds=4096$saltsaltsal$L7gFZEqSLEq9mNwsKNtA7.F3dXQdpI4jHKEFLn3M7VTdk
```

```
# Install required packages
```

```
packages:
```

```
- ubuntu-desktop-minimal
```

```
- firefox
```

```
- xrdp
```

```
- xserver-xorg-video-dummy
```

```
- net-tools
```

```
- openssh-server
```

```
- inotify-tools
```

```
# Run commands after boot
```

```
runcmd:
```

```
- systemctl enable xrdp
```

```
- systemctl start xrdp
```

```
- adduser xrdp ssl-cert
```

```
- ufw allow 3389/tcp
```

```
- echo "firefox" > /home/browser/.xsession
```

```
- chmod +x /home/browser/.xsession
```

```
- chown browser:browser /home/browser/.xsession
```

```
# Set root password
```

```
chpasswd:
```

```
list: |
```

```
  root:browser123
```

```
  browser:browser123
```

```
expire: False
```

```
# Auto-resize filesystem
growpart:
  mode: auto
  devices: ['/']
EOF

# Create meta-data file
cat > meta-data << 'EOF'
instance-id: browser-isolation-001
local-hostname: browser-isolation
EOF

# Create network-config (optional)
cat > network-config << 'EOF'
version: 2
ethernets:
  enp0s3:
    dhcp4: true
EOF

# Generate ISO for cloud-init
cloud-localds cloud-init.img user-data meta-data

echo "✅ Cloud-init configuration created"
```

Part 4: Set Up Networking

Step 4.1: Create TAP Device

```
bash
```

```
# Create network setup script
cat > ~/cloud-hypervisor-setup/setup-network.sh << 'EOF'
#!/bin/bash

# Create TAP device
sudo ip tuntap add tap-ch0 mode tap
sudo ip addr add 192.168.100.1/24 dev tap-ch0
sudo ip link set tap-ch0 up

# Enable IP forwarding
sudo sysctl -w net.ipv4.ip_forward=1

# Get main network interface
MAIN_IF=$(ip route | grep default | awk '{print $5}')

# Set up NAT
sudo iptables -t nat -A POSTROUTING -o $MAIN_IF -j MASQUERADE
sudo iptables -A FORWARD -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
sudo iptables -A FORWARD -i tap-ch0 -o $MAIN_IF -j ACCEPT

echo "✅ Network configured: tap-ch0 at 192.168.100.1"
echo "    VM will get IP via DHCP (likely 192.168.100.2)"
EOF

chmod +x setup-network.sh

# Run network setup
./setup-network.sh
```

Part 5: Configure Cloud Hypervisor

Step 5.1: Create VM Configuration File

```
bash
```

```
cd ~/cloud-hypervisor-setup
```

```
cat > vm-config.json << 'EOF'
```

```
{
  "cpus": {
    "boot_vcpus": 2,
    "max_vcpus": 2
  },
  "memory": {
    "size": 2147483648
  },
  "kernel": {
    "path": "/boot/vmlinuz"
  },
  "cmdline": {
    "args": "console=hvc0 root=/dev/vda1 rw"
  },
  "disks": [
    {
      "path": "/home/YOUR_USERNAME/cloud-hypervisor-setup/ubuntu-browser.img"
    },
    {
      "path": "/home/YOUR_USERNAME/cloud-hypervisor-setup/cloud-init.img",
      "readonly": true
    }
  ],
  "net": [
    {
      "tap": "tap-ch0",
      "mac": "12:34:56:78:90:ab",
      "ip": "192.168.100.2",
      "mask": "255.255.255.0"
    }
  ],
  "console": {
    "mode": "Tty"
  },
  "serial": {
    "mode": "Null"
  }
}
```

```
EOF
```

```
# Replace YOUR_USERNAME
```

```
sed -i "s|YOUR_USERNAME|$USER|g" vm-config.json
```

Step 5.2: Create Launcher Script (Method 1: VNC)

```
bash
```



```
cat > ~/cloud-hypervisor-setup/launch-vnc.sh << 'EOF'
#!/bin/bash

echo "🚀 Starting Cloud Hypervisor with VNC..."

# Ensure network is ready
./setup-network.sh

# Start Cloud Hypervisor
cloud-hypervisor \
  --cpus boot=2 \
  --memory size=2G \
  --disk path=ubuntu-browser.img \
  --disk path=cloud-init.img,readonly=on \
  --net tap=tap-ch0 \
  --console off \
  --serial tty &

CH_PID=$!

echo "⌚ Waiting for VM to boot (60 seconds)..."
sleep 60

# Get VM IP
VM_IP="192.168.100.2"

echo "🌐 VM booted at: $VM_IP"
echo "   Connecting via VNC..."

# SSH into VM and start VNC server
ssh-keygen -f ~/.ssh/known_hosts -R $VM_IP 2>/dev/null

# Install and start x11vnc
sshpass -p 'browser123' ssh -o StrictHostKeyChecking=no browser@$VM_IP << 'REMOTE'
# Install VNC server if not present
sudo apt install -y x11vnc

# Start X server
export DISPLAY=:0
sudo X :0 -config /etc/X11/xorg.conf &
sleep 3

# Start VNC server
x11vnc -display :0 -forever -shared -nopw &

# Start Firefox
```

```
DISPLAY=:0 firefox &
REMOTE

# Open VNC viewer
sleep 5
vncviewer $VM_IP:5900 &

echo "✅ Cloud Hypervisor browser ready!"
echo "    PID: $CH_PID"
echo "    Press Ctrl+C to stop"

wait $CH_PID
EOF

chmod +x launch-vnc.sh
```

Step 5.3: Create Launcher Script (Method 2: SPICE)

```
bash
```

```
cat > ~/cloud-hypervisor-setup/launch-spice.sh << 'EOF'
#!/bin/bash

echo "🚀 Starting Cloud Hypervisor with SPICE..."

# Note: Cloud Hypervisor has limited SPICE support
# Using VNC-based approach with virt-viewer instead

./setup-network.sh

# Start VM
cloud-hypervisor \
  --cpus boot=2 \
  --memory size=2G \
  --disk path=ubuntu-browser.img \
  --disk path=cloud-init.img,readonly=on \
  --net tap=tap-ch0 \
  --console off &

CH_PID=$!

echo "⌚ VM starting..."
sleep 60

VM_IP="192.168.100.2"

# Connect with RDP (xrdp pre-configured in cloud-init)
echo "🌐 Connecting via RDP (xrdp)..."
remmina -c rdp://browser:browser123@$VM_IP &

echo "✅ Connected!"
wait $CH_PID
EOF

chmod +x launch-spice.sh
```

Part 6: Launch Virtual Machine

Step 6.1: First Boot

```
bash
```

```
cd ~/cloud-hypervisor-setup

# Ensure network is setup
./setup-network.sh

# Start Cloud Hypervisor manually for first boot
sudo cloud-hypervisor \
  --cpus boot=2 \
  --memory size=2G \
  --disk path=ubuntu-browser.img \
  --disk path=cloud-init.img,readonly=on \
  --net tap=tap-ch0,mac=12:34:56:78:90:ab \
  --console tty \
  --serial null &

# Wait for boot (watch console)
# Login with: browser / browser123
```

Step 6.2: Configure Display Inside VM

```
bash
```

```
# SSH into VM (after boot)
ssh browser@192.168.100.2
# Password: browser123

# Inside VM - Install additional packages
sudo apt update
sudo apt install -y \
    xfce4 \
    xfce4-goodies \
    tightvncserver \
    firefox

# Configure VNC server
vncserver
# Set password when prompted

# Kill VNC for now
vncserver -kill :1

# Create startup script
cat > ~/.vnc/xstartup << 'EOF'
#!/bin/bash
xrdp $HOME/.Xresources
startxfce4 &
EOF

chmod +x ~/.vnc/xstartup

# Start VNC permanently
vncserver -geometry 1920x1080 -depth 24

# Exit SSH
exit
```

Part 7: Connect to Browser

Step 7.1: VNC Connection

```
bash
```

```
# From host machine
vncviewer 192.168.100.2:5901

# Should see XFCE desktop with Firefox available
```

Step 7.2: RDP Connection (Alternative)

```
bash

# Install RDP client if not present
sudo apt install -y remmina remmina-plugin-rdp

# Connect via RDP (xrdp is configured in cloud-init)
remmina -c rdp://browser:browser123@192.168.100.2
```

Part 8: File Monitoring

Step 8.1: Monitor Script (Host Side)

```
bash
```

```
cat > ~/cloud-hypervisor-setup/file-monitor.py << 'EOF'
#!/usr/bin/env python3
"""
Monitor file operations in Cloud Hypervisor VM
"""

import paramiko
import time
import logging

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)

def monitor_vm():
    vm_ip = "192.168.100.2"
    username = "browser"
    password = "browser123"
    watch_path = "/home/browser/Downloads"

    ssh = paramiko.SSHClient()
    ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())

    try:
        ssh.connect(vm_ip, username=username, password=password)
        logging.info(f"👁 Connected to VM: {vm_ip}")

        # Start inotify watch
        cmd = f"inotifywait -m {watch_path} -e create -e modify -e delete"
        stdin, stdout, stderr = ssh.exec_command(cmd)

        for line in stdout:
            logging.warning(f"🚨 FILE EVENT: {line.strip()}")

    except KeyboardInterrupt:
        logging.info("Monitoring stopped")
    finally:
        ssh.close()

if __name__ == "__main__":
    # Install: pip3 install paramiko
    monitor_vm()
EOF

chmod +x file-monitor.py
```

```
# Install dependency
pip3 install paramiko
```

Part 9: Desktop Integration

Step 9.1: Create Desktop Launcher

```
bash

cat > ~/.local/share/applications/cloud-hypervisor-browser.desktop << EOF
[Desktop Entry]
Version=1.0
Type=Application
Name=Isolated Browser (Cloud Hypervisor)
Comment=Launch browser in Cloud Hypervisor microVM
Exec=$HOME/cloud-hypervisor-setup/launch-vnc.sh
Icon=web-browser
Terminal=true
Categories=Network;WebBrowser;Security;
EOF

update-desktop-database ~/.local/share/applications/

echo "✅ Desktop launcher created"
```

Part 10: Performance Benchmarking

Step 10.1: Boot Time Test

```
bash
```



```
cat > ~/cloud-hypervisor-setup/benchmark.sh << 'EOF'
#!/bin/bash

echo "🕒 Benchmarking Cloud Hypervisor..."

./setup-network.sh

START=$(date +%s%3N)

cloud-hypervisor \
  --cpus boot=2 \
  --memory size=2G \
  --disk path=ubuntu-browser.img \
  --net tap=tap-ch0 \
  --console off &

CH_PID=$!

# Wait for network response
while ! ping -c 1 192.168.100.2 &>/dev/null; do
  sleep 0.1
done

END=$(date +%s%3N)
BOOT_TIME=$((END - START))

echo "✅ Cloud Hypervisor boot time: ${BOOT_TIME}ms"

kill $CH_PID
EOF

chmod +x benchmark.sh
```

Configuration Files Summary

```
cloud-hypervisor-setup/
├─ ubuntu-browser.img      # VM disk image
├─ cloud-init.img          # Cloud-init configuration
├─ vm-config.json          # VM configuration
├─ user-data               # Cloud-init user data
├─ meta-data               # Cloud-init metadata
├─ setup-network.sh        # Network setup
├─ launch-vnc.sh           # VNC launcher
```

```
|— launch-spice.sh           # RDP launcher
|— file-monitor.py          # File monitoring
|— benchmark.sh             # Performance test
```

Quick Start Commands

```
bash

# 1. First-time setup
cd ~/cloud-hypervisor-setup
./setup-network.sh

# 2. Launch with VNC
./launch-vnc.sh

# 3. Or launch with RDP
./launch-spice.sh

# 4. Monitor files (separate terminal)
python3 file-monitor.py

# 5. Benchmark
./benchmark.sh
```

Expected Performance

- **VM Boot Time:** 100-150ms (microVM init)
 - **Total to Browser:** 2-3 seconds
 - **Memory Usage:** ~2GB RAM
 - **Display Quality:** Better than Firecracker (less compression)
-

Advantages & Disadvantages

✅ Advantages:

- Own kernel (hardware isolation)
- Better display support than Firecracker
- GPU support via virtio-gpu

- Active development
- Similar security to Firecracker

✗ Disadvantages:

- Less mature than QEMU
 - Smaller community
 - Still requires VNC/RDP setup
 - Linux host only
-

Troubleshooting

VM won't boot:

```
bash

# Check KVM access
ls -l /dev/kvm

# Check network
ip addr show tap-ch0

# View VM console
cloud-hypervisor --console tty ...
```

Can't connect via VNC:

```
bash

# SSH into VM
ssh browser@192.168.100.2

# Check VNC status
ps aux | grep vnc
netstat -tulpn | grep 5900

# Restart VNC
vncserver -kill :1
vncserver -geometry 1920x1080
```

This completes Cloud Hypervisor setup! 🎉